# Data Science

## Lecture 10: Projections and Dimensionality Reduction

Prof. Dr. Michael Granitzer

Speaker notes

Converted from LNKB-tex part-en-ds-chp9-visual-analytics-projections.tex. Builds on Lecture 02 (variance, covariance), Lecture 07 (curse of dimensionality) and Lecture 08 (distance matrices).

# Where We Left Off

Lectures 08–09 found structure in more dimensions than anyone can look at. Today maps them into two — which requires deciding what to lose.

**Lecture 07**
plot it

→

**Lectures 08–09**
too many dimensions to plot

→

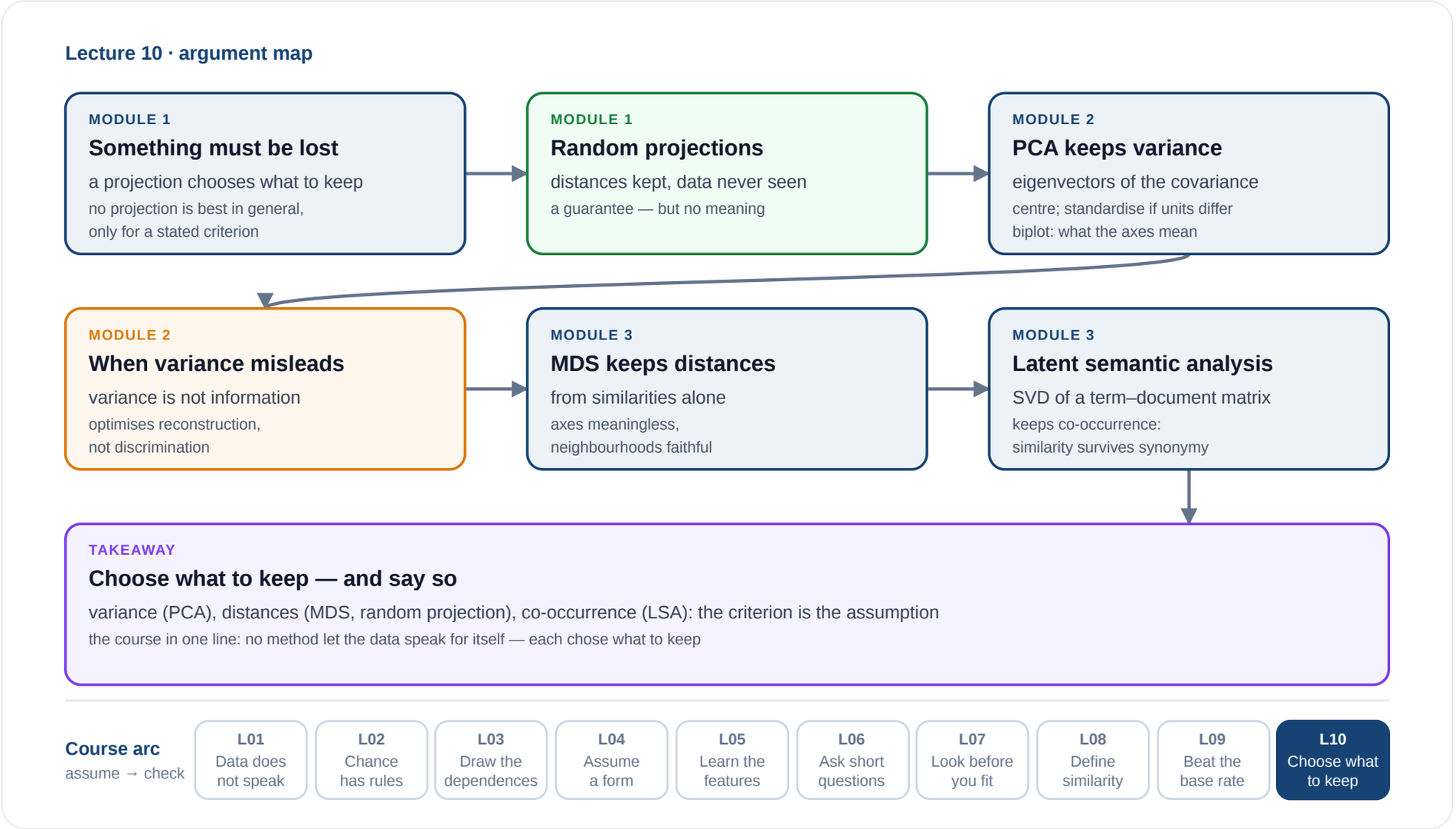
**Today**
project: choose what to keep

> This is the last lecture — and the last instance of Lecture 01's claim: no projection lets the data speak for itself.

Speaker notes

Lecture 07's advice was "plot before you fit". With four attributes that is a scatterplot matrix; with forty it is impossible. A projection makes the plot possible again, at the price of a criterion for what the picture keeps. Every method today is a different criterion.

# Argument Map



Speaker notes

Something must be lost; a projection is defined by what it keeps. Random projections keep distances without looking at the data: a guarantee without meaning. PCA keeps variance: powerful where variance is information, misleading where it is not. MDS keeps distances from similarities alone; LSA keeps co-occurrence. End on the bottom row and then on the whole strip: every lecture chose what to keep; the data never chose for us.

# Module 1

## What a Projection Preserves

> ***Guiding Question:*** *Reducing dimensions must lose information. Which information do you choose to keep?*

# Seeing High-Dimensional Data

A **geometric projection** maps high-dimensional data onto a low-dimensional space — usually 2-D or 3-D, so that it can be plotted and reasoned about.

## Linear projections

$$\mathbf{z} = \mathbf{P}\mathbf{x}$$

A matrix $\mathbf{P}$ maps every point. Distortion is global and predictable, and each new axis is a weighted sum of the old ones. A *reducing* $\mathbf{P}$ is not invertible: what it drops cannot be recovered.

## Non-linear projections

No single matrix. Methods such as **t-SNE** and **UMAP** keep **local neighbourhoods** — who is near whom — at the price of global distances. Often more faithful visually, harder to interpret.

**The central question of the chapter:** how do we find the *best* projection — and by which criterion is one projection better than another?

Speaker notes

Guiding question (P1). Every method that follows answers it differently: random projection says "any projection, with a guarantee", PCA says "the one keeping most variance", MDS says "the one preserving distances". Naming the criterion is naming the method.

# What Is Necessarily Lost

Reducing dimensions **must** lose information — the question is only which.

### Variance

PCA keeps as much of it as a linear map can.

### Distances

MDS keeps pairwise distances as faithfully as possible.

### Guarantees

Random projection keeps distances within a provable error, cheaply.

A projection is defined by **what it chooses to preserve**. There is no best-in-general projection, only a best one for a stated criterion.

# The Surprising Result

**Johnson–Lindenstrauss lemma.** For any $0 < \epsilon < 1$ and $n$ points in any dimension, a projection onto

$$d = O\left(\frac{\log n}{\epsilon^2}\right)$$

dimensions exists that preserves all pairwise distances to within a factor $1 \pm \epsilon$.

The target dimension depends on the **number of points** and on the accuracy wanted — **not on the original dimensionality**. A million-dimensional dataset and a thousand-dimensional one need the same target dimension.

Speaker notes

This is genuinely counterintuitive and worth pausing on. It says high-dimensional data is "thinner" than it looks: n points span at most n−1 dimensions, and approximate distances need far fewer.

# Construction

1. Draw a $d \times p$ matrix $\mathbf{R}$ with entries sampled independently (Gaussian, or $\pm 1$ with equal probability).
2. Project: $\mathbf{z} = \frac{1}{\sqrt{d}}\mathbf{R}\mathbf{x}$.

With $d$ as in the lemma, such a random $\mathbf{R}$ meets the $1 \pm \epsilon$ bound **with high probability** — the lemma's projection can simply be drawn.

# Properties and Limits

## Properties

No training, no data pass — the matrix does not depend on the data at all. Extremely fast, trivially applied to new points, and it comes with a bound.

## Limits

The axes are meaningless, so the result is not interpretable. It preserves distances, not structure, and works best when $d$ is not too small.

Random projection preserves distances **with a guarantee and without looking at the data**. Use it when you need speed and a bound; use PCA when you need meaning.

# Module 2

## Principal Component Analysis

> ***Guiding Question:*** *PCA keeps the directions of largest variance. When is variance the wrong thing to keep?*

# Which Direction Matters?

A cloud stretched along a diagonal: projected onto the **short** axis it collapses into a blob; onto the **long** axis its points stay apart.

**PCA's criterion:** the 1st component is the direction of **maximum variance**; each further one takes the most *remaining* variance, **orthogonal** to all before it.

The first $m$ components are the **best $m$-dimensional linear approximation**: most variance kept = least squared reconstruction error.

PCA keeps the directions of highest **variance**. Its assumption — variance is information — makes it powerful, and makes it fail.

Speaker notes

Intuition first (P4). The equivalence in the note is Pythagoras: for centred data, each point's squared length splits into the part kept by the projection and the part lost, so maximising one minimises the other. Ask whether "most variance" always means "most interesting" — it does not: for classification, the discriminating direction may have small variance. That caveat returns in the mini-exercise of the algorithm unit.

# Covariance Is the Object to Diagonalise

For centred data $\mathbf{X}$, the **covariance matrix**

$$\mathbf{C} = \frac{1}{n-1}\mathbf{X}^\top\mathbf{X}$$

holds the variance of each attribute on its diagonal and the covariance of each pair off it.

### Large diagonal entry

That attribute varies a lot — potentially informative.

### Large off-diagonal entry

Two attributes are **redundant**: one partly predicts the other.

The goal: find new axes in which the covariance matrix is **diagonal** — no redundancy left between components — and order them by variance.

Speaker notes

Framing PCA as "remove redundancy, keep variation" makes the covariance matrix the natural object. It also connects directly to the joint-distribution unit from Lecture 02, where covariance was introduced.

# The Solution

The directions that diagonalise $\mathbf{C}$ are its **eigenvectors**; the variance captured by each is the corresponding **eigenvalue**.

Sorting eigenvectors by descending eigenvalue gives the principal components, and the projection matrix $\mathbf{W}_m$ holds the top $m$ of them.

$\mathbf{C}$ is symmetric, so its eigenvectors are orthogonal: the components are **uncorrelated by construction**. The share of total variance in component $i$ is its **explained variance** $\lambda_i / \sum_j \lambda_j$.

Speaker notes

The explained-variance ratio is the practical output students will actually use — it is what a scree plot shows and how "how many components?" gets answered.

# A Two-Attribute Example

Two attributes in the same unit, each with variance 2, covariance 1:

$$\mathbf{C} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad \mathbf{C}\begin{pmatrix} 1 \\ 1 \end{pmatrix} = 3\begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \mathbf{C}\begin{pmatrix} 1 \\ -1 \end{pmatrix} = 1\begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

| component | direction | eigenvalue | explained variance |
|---|---|---|---|
| PC1 | $(1,1)/\sqrt{2}$ — "both together" | $\lambda_1 = 3$ | $3/4 = 75\%$ |
| PC2 | $(1,-1)/\sqrt{2}$ — "one against the other" | $\lambda_2 = 1$ | $1/4 = 25\%$ |

PCA is the **eigendecomposition of the covariance matrix**: eigenvectors give the directions, eigenvalues give their importance, and the top ones are kept.

Speaker notes

Check by multiplication, no characteristic polynomial needed: C·(1,1) = (3,3), C·(1,−1) = (1,−1). The eigenvalues sum to the trace, 2 + 2 = 4, which is the total variance — that is why the shares are 3/4 and 1/4. Keeping only PC1 loses a quarter of the variance; the positive covariance is what tilts PC1 onto the diagonal. With covariance 0 both eigenvalues would be 2 and no direction would be preferred.

# The Procedure

1. **Centre** the data; **standardise** it if the units differ.
2. Compute the covariance matrix $\mathbf{C}$.
3. Compute its eigenvectors and eigenvalues.
4. Sort by descending eigenvalue; keep the top $m$.
5. Project: $\mathbf{Z} = \mathbf{X}\mathbf{W}_m$.

**Centring is not optional**: the covariance formula assumes it. **Standardising is a choice** — make it when units differ. Otherwise an attribute in metres dominates one in kilometres purely through its **unit**, and PCA reports the choice of unit as the principal direction.

Speaker notes

Also known as the discrete Karhunen–Loève transform. The standardisation trap is the single most common practical error and worth stating as a rule: always ask what the units are before running PCA. When all attributes share a unit (pixel intensities, say), standardising can hurt: it inflates directions that are mostly noise.

# Choosing the Number of Components

A **scree plot** shows eigenvalues in descending order; the usual reading is the same elbow heuristic as for choosing $k$ in clustering, or a cumulative explained-variance threshold (80%, 95%).

### Benefits

Removes correlated and noisy directions; speeds up downstream methods; makes 2-D visualisation possible; components are uncorrelated by construction.

### Drawbacks

Components are **linear combinations of all attributes**, so interpretability is lost. Only **linear** structure is captured. And variance is assumed to equal importance — which for classification can be false.

# Mini-Exercise

You run PCA on the standardised iris data; the first two components explain 96% of the variance.

**Are these two components therefore good features for telling the species apart?**

# Mini-Exercise

You run PCA on the standardised iris data; the first two components explain 96% of the variance.

**Are these two components therefore good features for telling the species apart?**

**Not necessarily.** 96% says a 2-D picture loses only 4% of the variation. PCA never saw the labels: it optimises reconstruction, and the species may differ along a thin, low-variance direction.

# Mini-Exercise

You run PCA on the standardised iris data; the first two components explain 96% of the variance.

**Are these two components therefore good features for telling the species apart?**

**Not necessarily.** 96% says a 2-D picture loses only 4% of the variation. PCA never saw the labels: it optimises reconstruction, and the species may differ along a thin, low-variance direction.

> PCA is centre → covariance → eigendecomposition → keep the top components. It optimises **reconstruction**, not discrimination; centring is mandatory, standardising a choice made by the units.

# Points and Variables in One Picture

A **biplot** overlays two things on the same axes:

### The objects

Each data point, plotted by its scores on the first two components — an ordinary scatterplot of the projection.

### The variables

Each original attribute drawn as an **arrow**: its loading on the two components. Direction shows what the attribute contributes to, length shows how strongly.

A biplot is how you recover interpretability after PCA has destroyed it: the arrows say what the anonymous components *mean* in terms of the original attributes.

Speaker notes

Worth demonstrating on any two-component PCA: students can immediately name PC1 ("overall size") and PC2 ("shape") from the arrow pattern, which no eigenvalue table conveys.

# Reading the Arrows

| arrows | the two attributes are |
|---|---|
| pointing together | positively correlated |
| pointing in opposite directions | negatively correlated |
| at about 90° | roughly uncorrelated |
| long | well represented by the two components |
| short | mostly outside the plotted plane — do not read its angles |

The biplot answers "what do these axes mean?" by drawing the original attributes into the projected space — points and variables read together.

Speaker notes

The angle reading is exact only for the part of each attribute that lies in the plane of the first two components; on standardised data with most variance in two components it approximates the correlation well. A short arrow is the warning sign: that attribute lives mainly in the discarded components.

# The Churn Shop: What Does PC1 Mean?

Four customer attributes: tenure, annual spend, app logins, support tickets. A biplot of the standardised table might show:

### THE ARROWS

Tenure, spend and logins point **together** along PC1; tickets points the **other way**.

### THE READING

PC1 is something like **engagement**, and the customers who churned sit at its low end.

Speaker notes

The running example, one last time; the arrow pattern is illustrative, not computed from a real table. The name "engagement" is our interpretation of the arrows, not something PCA found: the method only keeps the direction of largest variance. Ask whether PC1 is the direction that best separates churners from stayers — not necessarily: PCA never saw the churn label (variance is not discrimination, as the map says).

# The Churn Shop: What Does PC1 Mean?

Four customer attributes: tenure, annual spend, app logins, support tickets. A biplot of the standardised table might show:

**THE ARROWS**

Tenure, spend and logins point **together** along PC1; tickets points the **other way**.

**THE READING**

PC1 is something like **engagement**, and the customers who churned sit at its low end.

Unstandardised, spend in euros has by far the largest variance, and PC1 is simply **spend**. Standardising decides what PC1 can mean — Lecture 08's scaling choice again.

# Module 3

## Beyond Variance

> ***Guiding Question:*** *Some objects have no coordinates — only similarities. Can they still be mapped?*

# A Different Criterion

PCA needs coordinates and preserves **variance**. MDS needs only a **dissimilarity matrix** and preserves **the distances themselves**.

Given pairwise dissimilarities $d_{ij}$, find low-dimensional points $\mathbf{z}_1, \ldots, \mathbf{z}_n$ minimising Kruskal's **stress-1**

$$S = \sqrt{\frac{\sum_{i<j} \left( \|\mathbf{z}_i - \mathbf{z}_j\| - d_{ij} \right)^2}{\sum_{i<j} d_{ij}^2}}$$

This is the projection method for objects that have **no coordinates at all** — only a dissimilarity: documents, molecules, survey respondents, perceived similarity ratings turned into distances.

Speaker notes

The denominator makes stress-1 scale-free: measuring in metres or kilometres gives the same value, so it can be compared across datasets (rules of thumb read 0.05 as good, 0.2 as poor). The distance-matrix-only requirement is the same property that made HAC applicable to non-vector data. Point at that parallel — it is the second time this course uses "all I need is a (dis)similarity".

# Classical MDS and Its Relatives

## Classical (metric) MDS

Solved in closed form by an eigendecomposition of the (double-centred) squared distance matrix. With Euclidean distances it is **equivalent to PCA**.

## Non-metric MDS

Preserves only the **rank order** of distances. Suited to ordinal similarity data — "A is more like B than like C" — where the numbers are not meaningful.

# Reading the Result

### Stress tells you the cost

A 2-D map of genuinely high-dimensional structure has high stress: the picture is a compromise, and the number says how big a one.

### Axes mean nothing

Unlike PCA components, MDS axes have no interpretation — only **relative positions** carry information. Rotations and reflections are equivalent.

MDS preserves **distances** rather than variance, works from a dissimilarity matrix alone, and produces a map whose axes are meaningless but whose **neighbourhoods are faithful** — as far as the stress allows.

Speaker notes

The "axes mean nothing" warning matters because students read MDS plots like scatterplots. Insist that only the pattern of proximity may be interpreted.

# The Problem With Matching Words

A document is represented as a vector of term counts — the **term–document matrix**. Similarity is then the overlap of vocabulary, which fails twice:

### Synonymy

"car" and "automobile" never match, though the documents are about the same thing.

### Polysemy

"bank" matches across finance and rivers, though the documents are unrelated.

> The vocabulary is the wrong space. We want a space of **concepts**, in which related terms lie close together.

Speaker notes

The two failures are the motivation for every embedding method that followed, right up to modern language models. Say so — students recognise the problem statement from tools they use.

# The Approach

Apply a **singular value decomposition** to the term–document matrix $\mathbf{A}$:

$$\mathbf{A} \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$$

and keep the $k$ largest singular values. Documents and terms are both represented in the resulting $k$-dimensional **latent space**.

SVD is PCA's underlying machinery applied to a rectangular matrix — the same "keep the strongest directions" idea, on term–document data (usually without centring).

# What the Latent Space Buys

### Dimension reduction

Thousands of terms collapse to a few hundred latent dimensions.

### Synonyms merge

Terms that co-occur with the same contexts end up close together.

### Noise drops

Rare, idiosyncratic usage lives in the discarded small singular values.

LSA projects documents and terms into a shared **latent space** via SVD, so that similarity survives synonymy. It is dimensionality reduction used as a representation, not just as compression.

Speaker notes

Connect forward honestly: modern embeddings solve the same problem with learned, non-linear, context-sensitive representations — but the objective LSA states is still recognisable in them.